

Hashing

Q.No:1

Problem Description

Everyone knows that some Pikachus despise becoming Raichus. (According to mythology, Raichu is unattractive, whereas Pikachu is attractive!)

How do we track down these unique Pikachus who despise evolution? Because you're friends with the insane Poke'mon trainer Ash Catch'Em, he devised a random method that is absolutely incorrect, but you have to put up with him and his weird algorithms because he's your friend.

He thinks if you are given N Pikachus in an array, $A_1, A_2 \dots A_N$, where each Pikachu is denoted by an integer. The total number of unique pairs (A_i, A_j) where $i < j$ is the number of Pikachus who hate evolution.

Constraints:

$$1 \leq N \leq 2 * 10^5$$

$$1 \leq A_i \leq 10^9$$

Input format:

The first line will consist of a single integer N . The second line consists of N integers $A_1, A_2 \dots A_N$.

Output format:

Output the total number of unique pairs (A_i, A_j) that can be formed, which will also be the number of special Pikachus.

Logical Test Cases

Test Case 1

INPUT (STDIN)

5
1 2 2 1 3

EXPECTED OUTPUT

6

Test Case 2

INPUT (STDIN)

7
1 4 1 2 2 1 3

EXPECTED OUTPUT

10

Mandatory Test Cases

Test Case 1

KEYWORD

for(i=0;i<n;i++)

Test Case 2

KEYWORD

if(arr[i]>max)

Code:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int first;
    int second;
} Pair;

int compare(const void *a, const void *b)
{
    Pair *p1 = (Pair *)a;
    Pair *p2 = (Pair *)b;
```

```
if (p1->first != p2->first)
return p1->first - p2->first;

return p1->second - p2->second;
}

int main()
{
int n;
scanf("%d", &n);

long long arr[n];

for(int i = 0; i < n; i++)
{
scanf("%lld", &arr[i]);
}

Pair *pairs = malloc((long long)n * n * sizeof(Pair));
long long count = 0;

for(int i = 0; i < n; i++)
{
for(int j = i + 1; j < n; j++)
{
pairs[count].first = arr[i];
pairs[count].second = arr[j];
count++;
}
}

qsort(pairs, count, sizeof(Pair), compare);

long long answer = 0;

for(long long i = 0; i < count; i++)
{
if(i == 0 ||
pairs[i].first != pairs[i - 1].first ||
pairs[i].second != pairs[i - 1].second)
{
answer++;
}
}

printf("Output:\n%lld\n", answer);

free(pairs);

return 0;
}
```

Output:

Test 1:

```
g++ 7graph/11
```

```
5
```

```
1 2 2 1 3
```

```
Output:
```

```
6
```

Test 2:

```
7
```

```
1 4 1 2 2 1 3
```

```
Output:
```

```
10
```